Course: 04733 AIOps **October 15, 2026**

Carnegie Mellon University Africa

Name: Martin Koome

Course: 04733 AIOps, Fall 2026

Lab 3

Real-Time Anomaly Detection with Prophet and Istio Metrics

1. Overview and Attachments

A Prophet model trained on normal Shipping service response times was used to flag anomalies in live Istio metrics. An *incident* is raised when the last four cycles are all anomalous. The monitor was first run locally, then deployed into the cluster with its flags published to Prometheus and Grafana, and finally generalized to cover every Boutique service. All source files required by the assignment are in the attached `Lab3_Attachments.zip`; screenshots appear in this report.

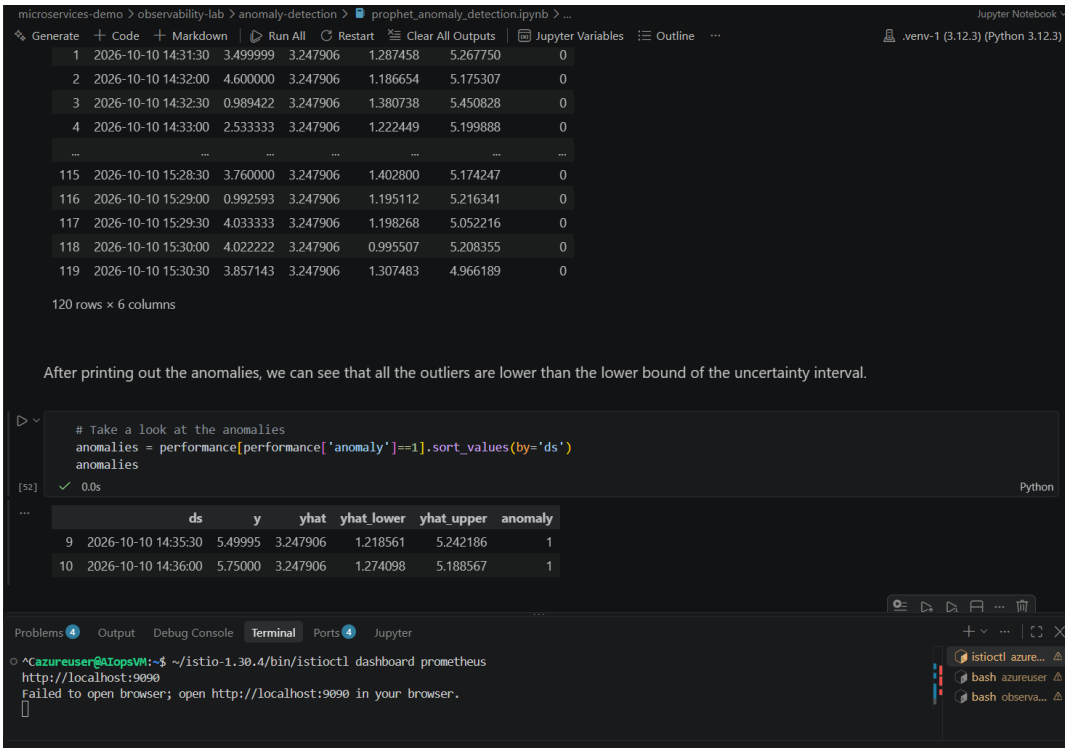
Table 1: Contents of `Lab3_Attachments.zip`, mapped to each task's required uploads.

Task	Required item	File in zip / location
Week 1 Task 1	curl query for training data	`Training_Data_collection_pipeline.txt`
	Training data pulled	`shipping_responsetime.json`
	Notebook anomaly result	Figure 1
Week 1 Task 2	Python monitor source	`monitor.py`
	Console output, no delay	Figure 2
Week 1 Task 3	Istio delay rule	`shippingservice-fault.yaml`
	Console output with delay	Figure 3
Week 2 Task 1	Dockerfile	`Dockerfile`
	Deployment YAML	`monitor_deploy.yaml`
	In-cluster console output	Figure 4
	Grafana dashboard	Figure 5
Week 2 Task 2	Generalized monitor	`monitor_all.py`
	Training-data script	`fetch_training_data.py`
	Updated Dockerfile	`Dockerfile All Services`
	Updated deployment YAML	`monitor_all_deploy.yaml`
	Istio delay rule	`03-delay-productcatalog.yaml`
	Updated Grafana dashboard	Figures 6 and 7

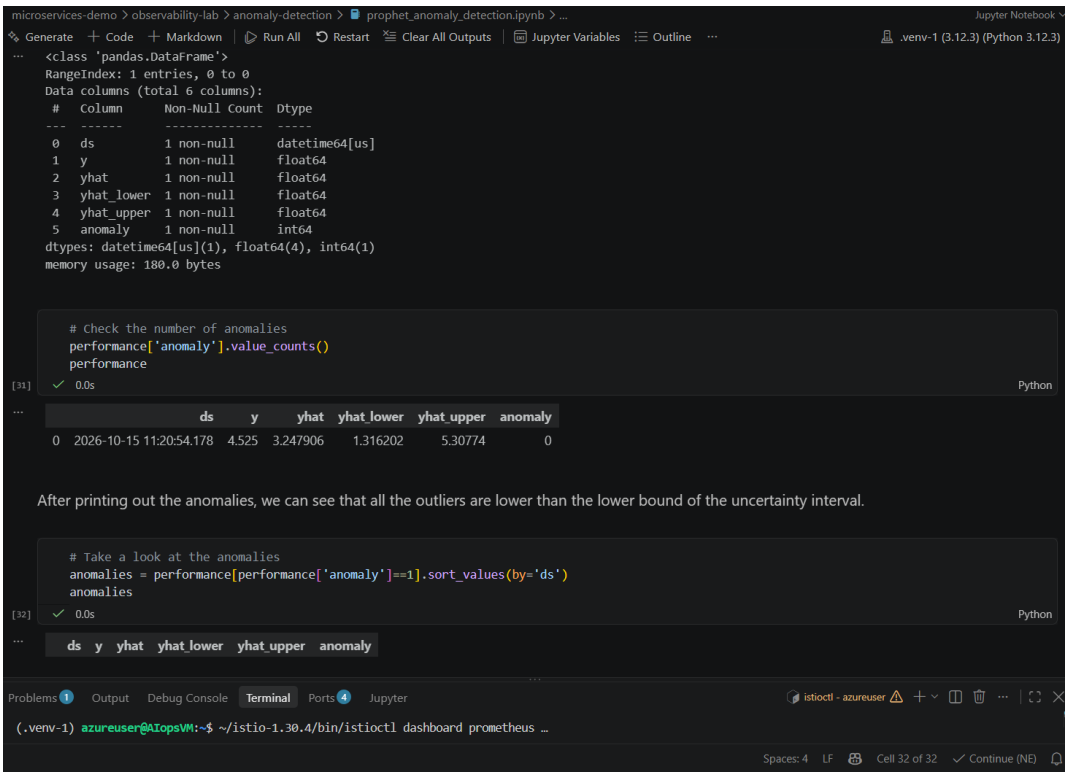
2. Week 1 Task 1 – Training Data and Notebook Check

With the load generator running, about 12 hours of normal traffic were pulled from Prometheus with `curl`. The query was the p95 `histogram_quantile` of `istio_request_duration_milliseconds_bucket`, summed over `le` and over both callers of `shippingservice`, using `reporter="source"` and a 30 s sub-query resolution. This gave 1440 points (09 Oct, 08:09:30–20:09:00), which were saved to `shipping_responsetime.json`.

In `prophet_demo.ipynb` the model, with its fixed flat trend, forecasts $\hat{y} \approx 3.25$ ms with an interval of roughly 1.2–5.3 ms (Figure 1). Over a one-hour live test (120 points), only two consecutive points (14:35:30 and 14:36:00) were flagged, both slightly above the upper bound. A single live point (4.53 ms) fell inside the interval and was not flagged. The training data therefore models normal behaviour well.



(a) Week 1 Task 1 – One-hour live test: 2 transient anomalies out of 120 points.



(b) Week 1 Task 1 – Single live data point: no anomaly.

Figure 1: Week 1 Task 1 – Anomaly results in the Prophet notebook using the collected training data.

3. Week 1 Task 2 – Monitor Under Normal Operation

`monitor.py` trains Prophet at startup and wakes every 60 s. On each cycle it pulls a single p95 value for the last minute, using the query below with no sub-query, and scores it as a one-row dataframe. It keeps a running dataframe of anomaly flags and sets `incident=1` only when the last four entries are all anomalous. Both values are published as one Prometheus gauge with `service` and `kind` labels on port 8000.

```
histogram_quantile(0.95, sum by (le) (increase(
  istio_request_duration_milliseconds_bucket{
    destination_canonical_service='shippingservice',
    reporter='source'}[1m])))
```

Listing 1: Week 1 Task 2: single-point query used by the monitor on each cycle

With no injected faults (Figure 2), every cycle from 11:48 to 11:53 stayed within the forecast interval (2.8–4.6 ms), so anomaly and incident remained 0.

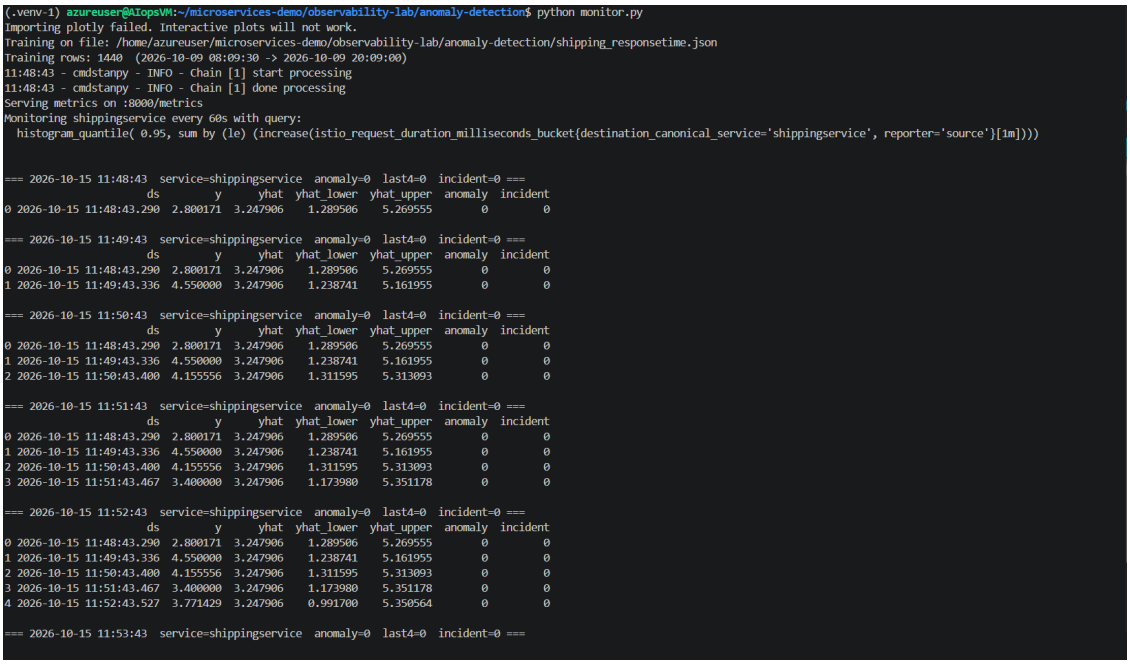


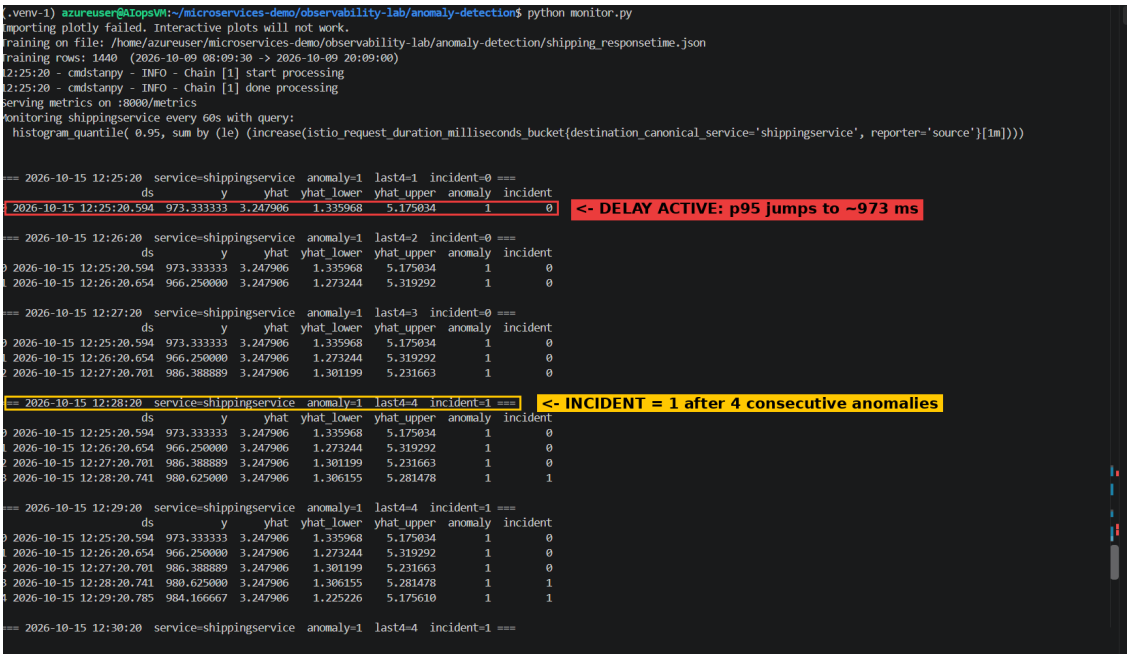
Figure 2: Week 1 Task 2 – Local monitor console output with no injected delays: no anomalies or incidents.

4. Week 1 Task 3 – Fault Injection and Detection

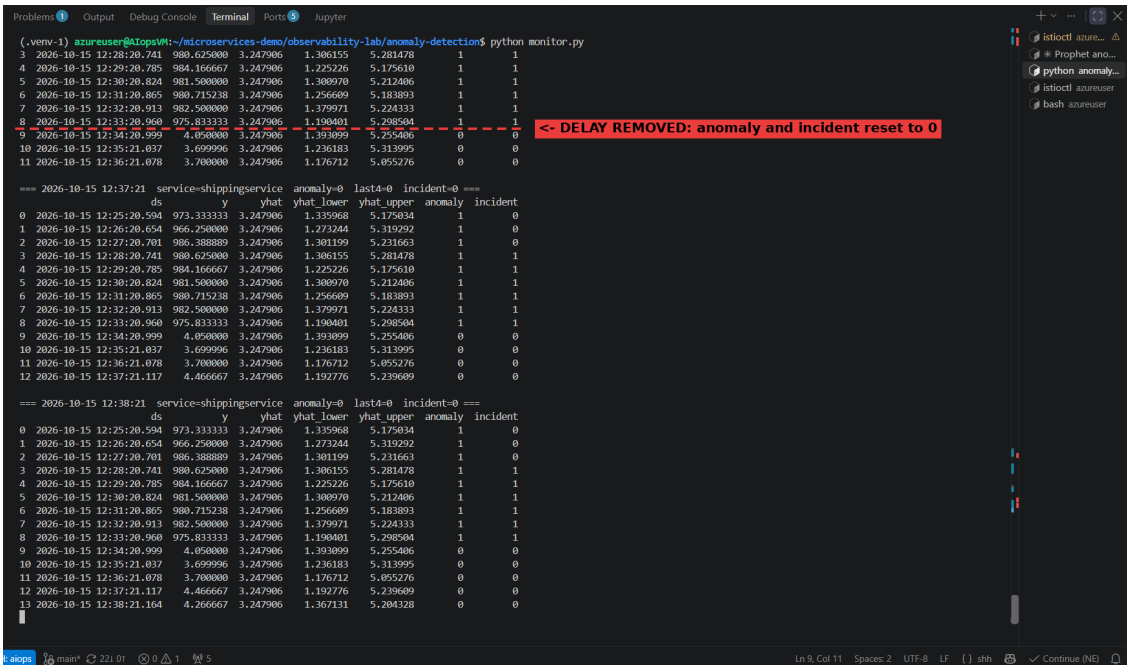
The Istio delay rule (`shippingservice-fault.yaml`) was applied as the monitor started, so the first cycle already shows its effect. It was deleted between the 12:33 and 12:34 cycles. The delay and removal points are marked in red in Figure 3; Table 2 summarizes the timeline.

Table 2: Week 1 Task 3 – Monitor timeline during the Istio delay test.

Cycle time	Event	p95 (ms)	anomaly	incident
12:25:20	Delay in effect (marked)	973	1	0
12:26:20–12:27:20	last4 = 2, 3	966–986	1	0
12:28:20	4th consecutive anomaly (marked)	981	1	**1**
12:29:20–12:33:20	Incident sustained	976–984	1	1
12:34:20	Delay removed (marked)	4.05	0	0
12:35:21–12:38:21	Normal operation	3.7–4.5	0	0



(a) Week 1 Task 3 – Part 1: delay detected and incident raised after 4 cycles.



(b) Week 1 Task 3 – Part 2: delay removed; anomaly and incident reset to 0.

Figure 3: Week 1 Task 3 – Local monitor console output with an injected 1 s Istio delay on `shippingservice`.

5. Week 2 Task 1 – Monitor Deployed in the Cluster

The monitor was built into a local Docker image with the training JSON copied in, and was deployed with `monitor_deploy.yaml`. For the in-cluster run, the Prometheus URL was changed to `prometheus.istio-system`. The `prometheus.io/scrape`, `port` and `path` annotations were added to the pod template so that Prometheus scrapes the gauges. Figure 4 shows the pod's console output in K9s: 18 normal cycles with no anomalies.

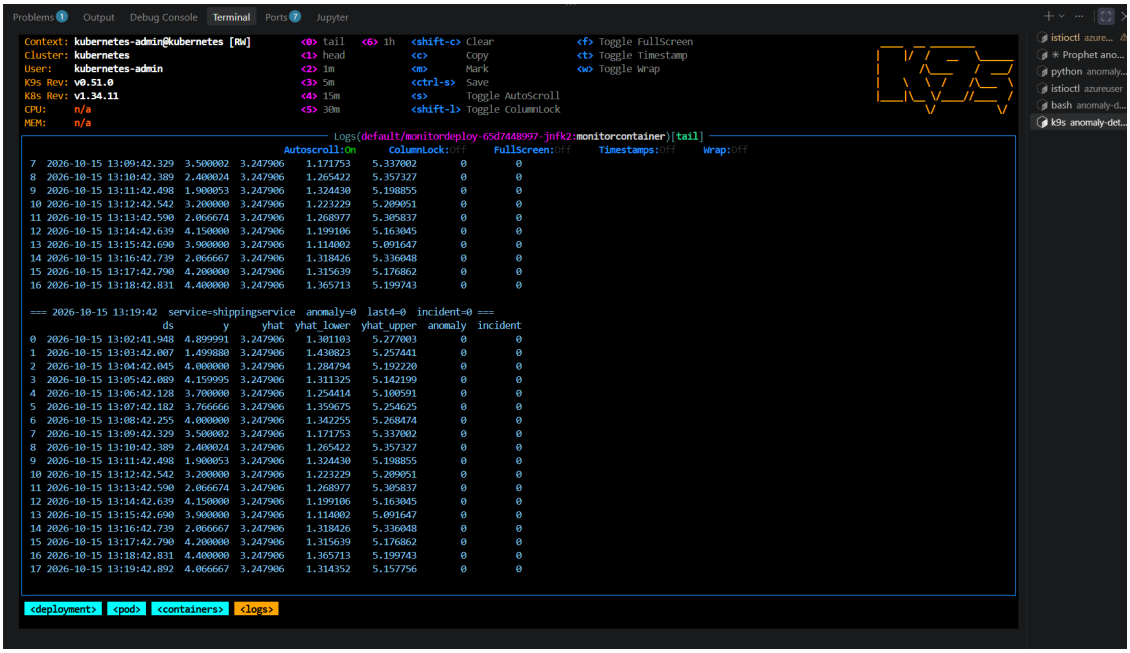
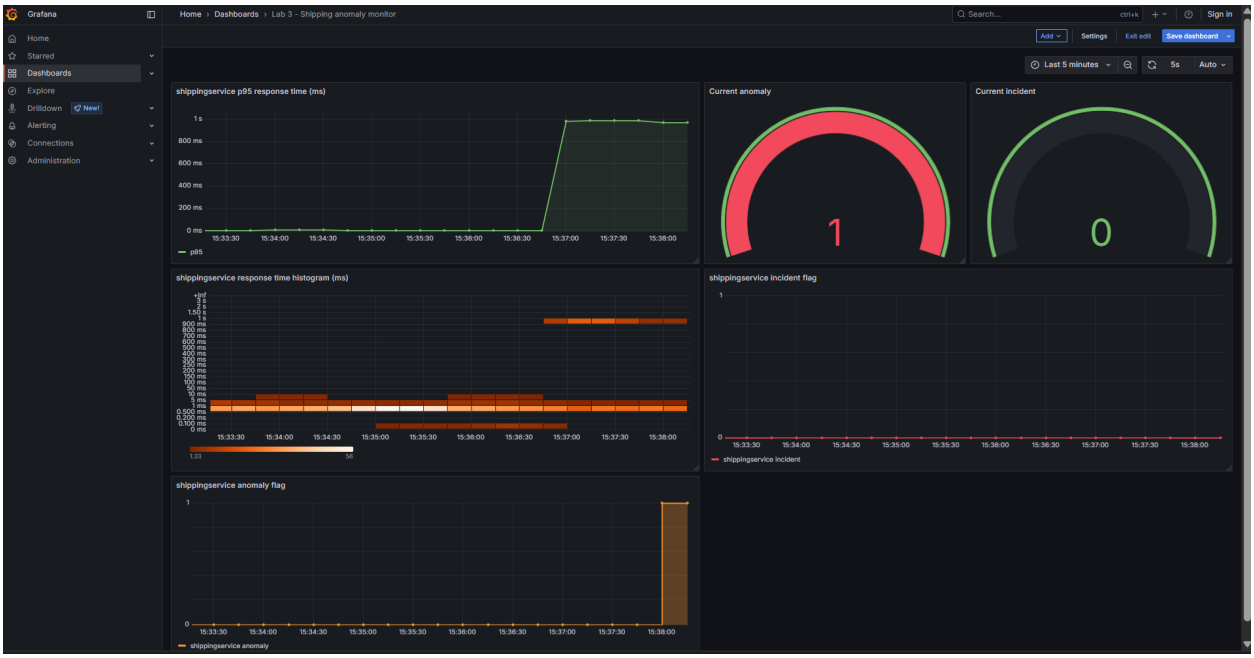
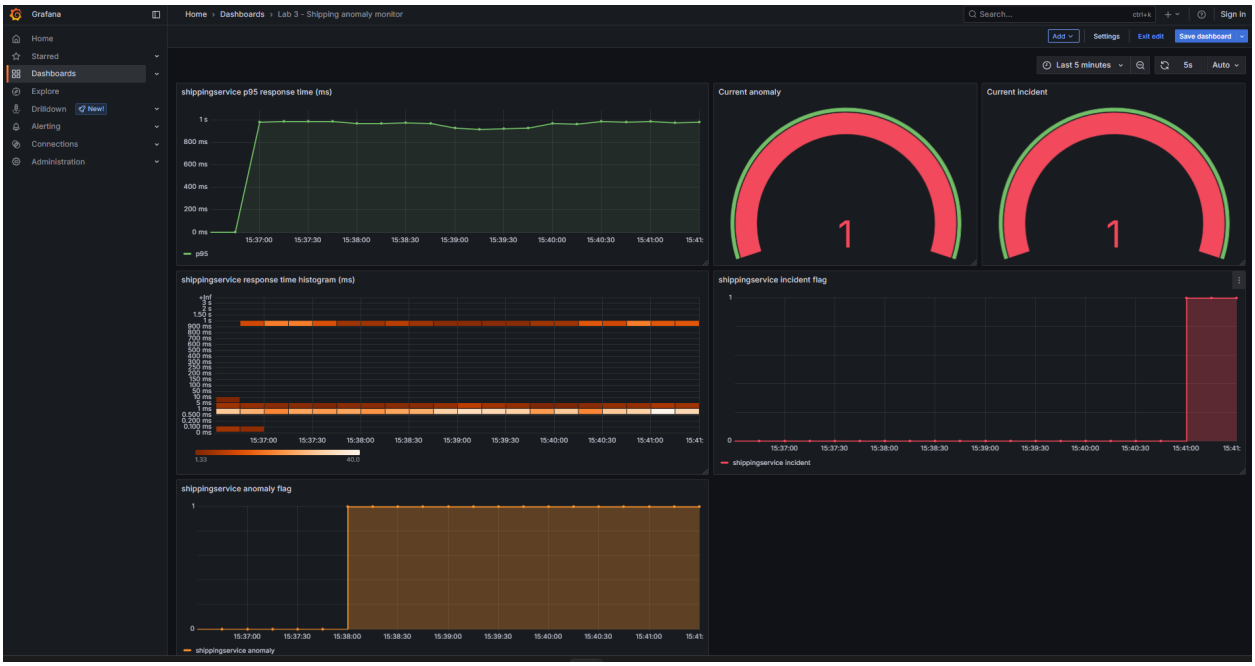


Figure 4: Week 2 Task 1 – Console output of the monitor pod (`monitordeploy/monitorcontainer`) running in the cluster.

The Grafana dashboard (Figure 5) shows the response-time histogram, the p95 time series, and the anomaly and incident flags as time series and as gauges. After the delay was injected at about 15:36:45, p95 rose to about 1 s and the anomaly flag went to 1 at the next cycle. The incident flag went to 1 about three minutes later, once four consecutive anomalies had accumulated.



(a) Week 2 Task 1 – Delay onset: anomaly = 1, incident still 0.



(b) Week 2 Task 1 – After 4 anomalous cycles: incident = 1.

Figure 5: Week 2 Task 1 – Grafana dashboard for the Shipping service monitor.

6. Week 2 Task 2 – Monitoring All Services

`fetch_training_data.py` pulls training data for every Boutique service, and `monitor_all.py` trains one Prophet model per service, publishing all flags under the same gauge. The rebuilt image was deployed with `monitor_all_deploy.yaml`.

The dashboard (Figures 6 and 7) has summary state timelines and a current-value table of all anomaly and incident flags, a p95 panel for all services, and detail rows for `shippingservice`, `checkoutservice` and `frontend`. Two delays are visible:

- **Shipping delay (about 15:37–15:49):** Shipping p95 rose to about 1 s and its flags were raised. Checkout p95 rose to about 2 s, consistent with checkout calling shipping twice per order.
- **Leaf-service delay on `productcatalogservice` (`03-delay-productcatalog.yaml`, from about 15:58):** anomaly and incident flags were raised for `productcatalogservice` and for all three of its callers: `frontend` (p95 about 4.9 s), `checkoutservice` (about 990 ms) and `recommendationservice` (about 568 ms). Unrelated services stayed at 0.

This confirms that a fault in a leaf service propagates as incidents to the services that depend on it.

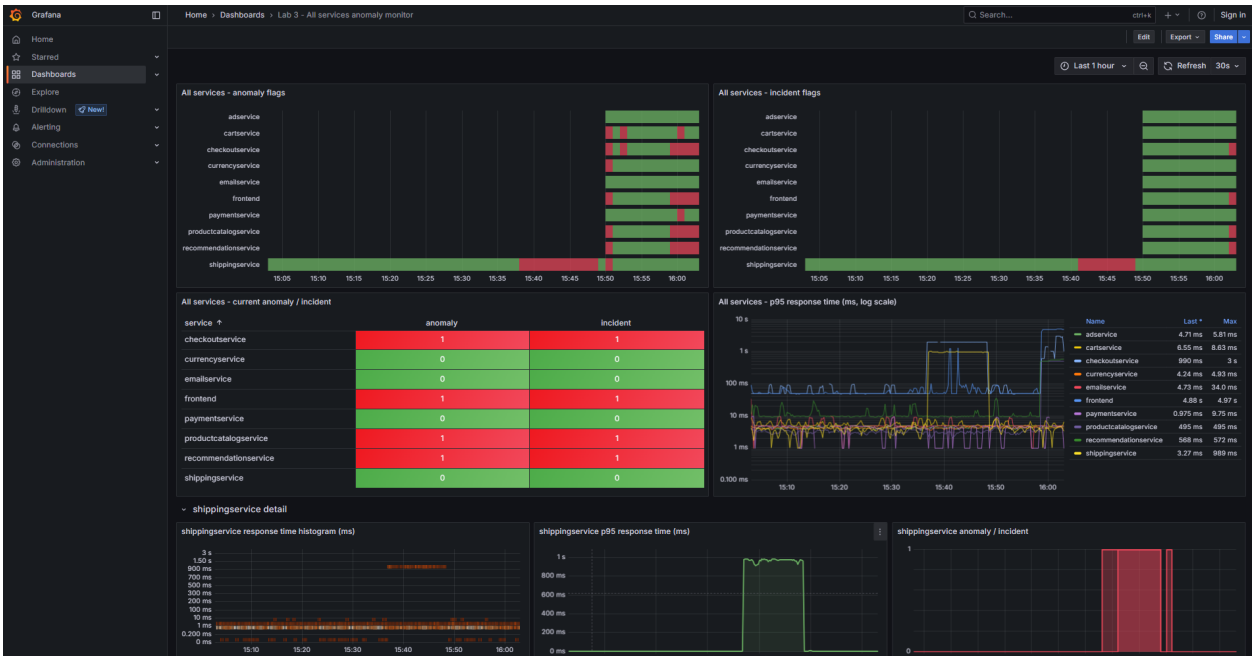


Figure 6: Week 2 Task 2 – All-services dashboard: summary of anomaly and incident flags, current values, and p95 for every service.

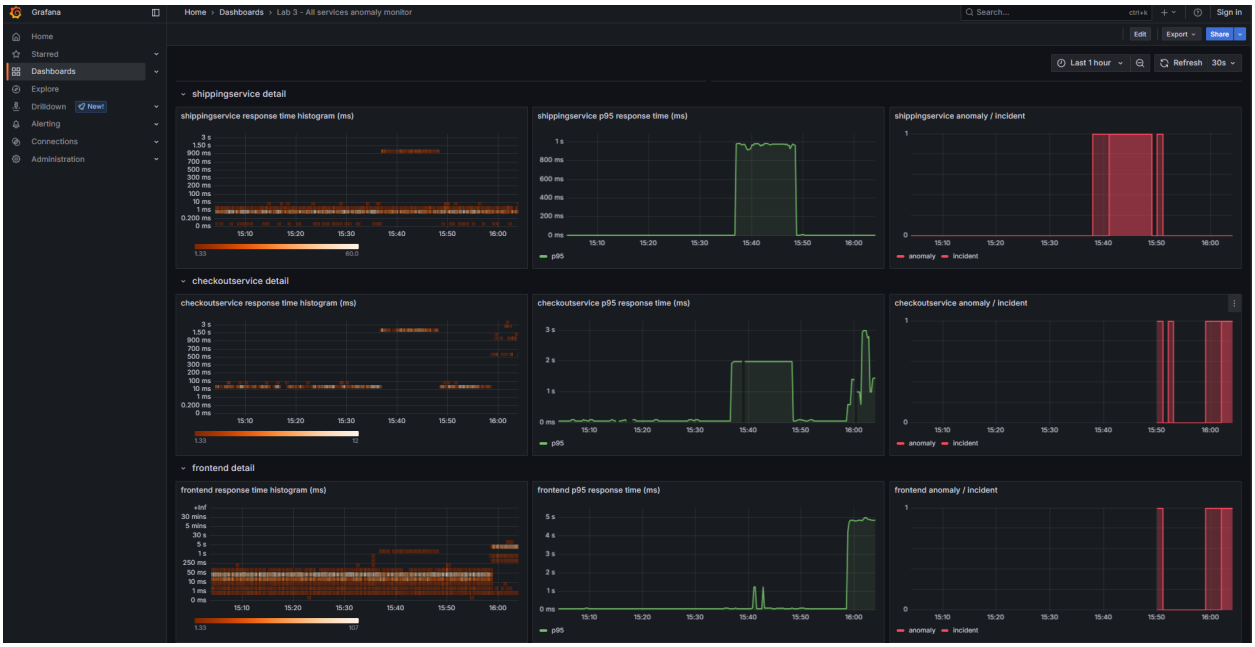


Figure 7: Week 2 Task 2 – Per-service detail for `shippingservice`, `checkoutservice` and `frontend`.

7. Conclusion

The Prophet monitor produced few false alarms under normal load and detected the injected delays within one cycle. Requiring four consecutive anomalies kept the incident flag from reacting to transients, and the flag cleared on the first normal cycle. Generalized to all services, the monitor showed how a single leaf-service fault surfaces as incidents across its dependents.